Enterprise Cloud Architecture & DevSecOps Reference Handbook

Target Audience: Cloud Architects, DevOps Engineers, Engineering Students
Scope: Three-Tier Architecture, Kubernetes Traffic Routing, Autoscaling Mechanics, End-to-End DevSecOps Pipelines (.NET & Java), Manifests, and Release Approval Governance.

Table of Contents

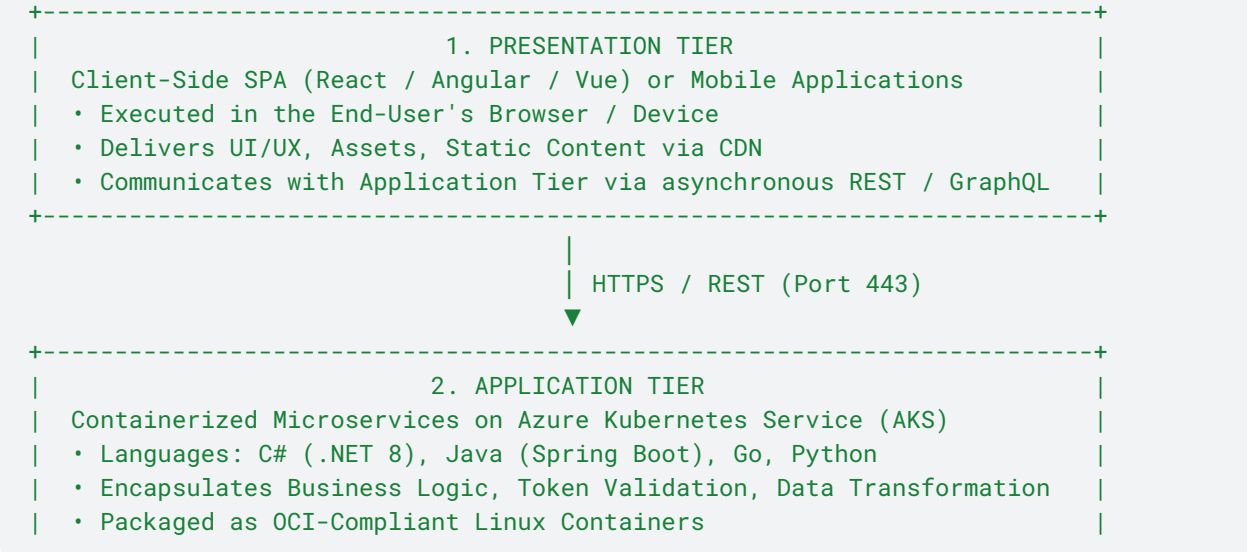
1. Module 1: Three-Tier Cloud Architecture & Request Journey (google.com/blog)
2. Module 2: Kubernetes Traffic Ingress & Pod Autoscaling Mechanics
3. Module 3: End-to-End DevSecOps Delivery Pipeline for .NET Applications
4. Module 4: End-to-End DevSecOps Delivery Pipeline for Java (Spring Boot) Applications
5. Module 5: Pipeline Release Governance: Approvals Framework

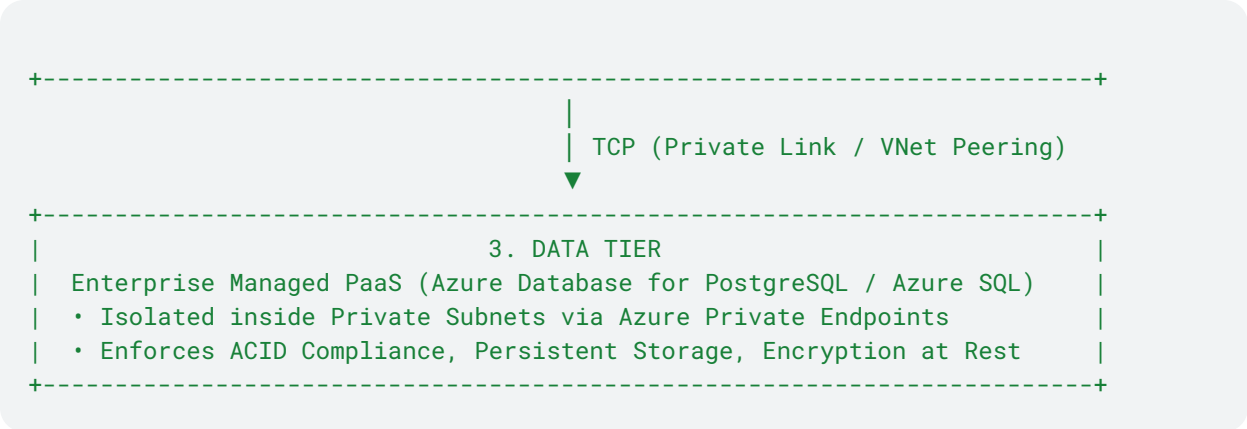
---

Module 1: Three-Tier Cloud Architecture & Request Journey

1.1 Decoupling the Three Tiers

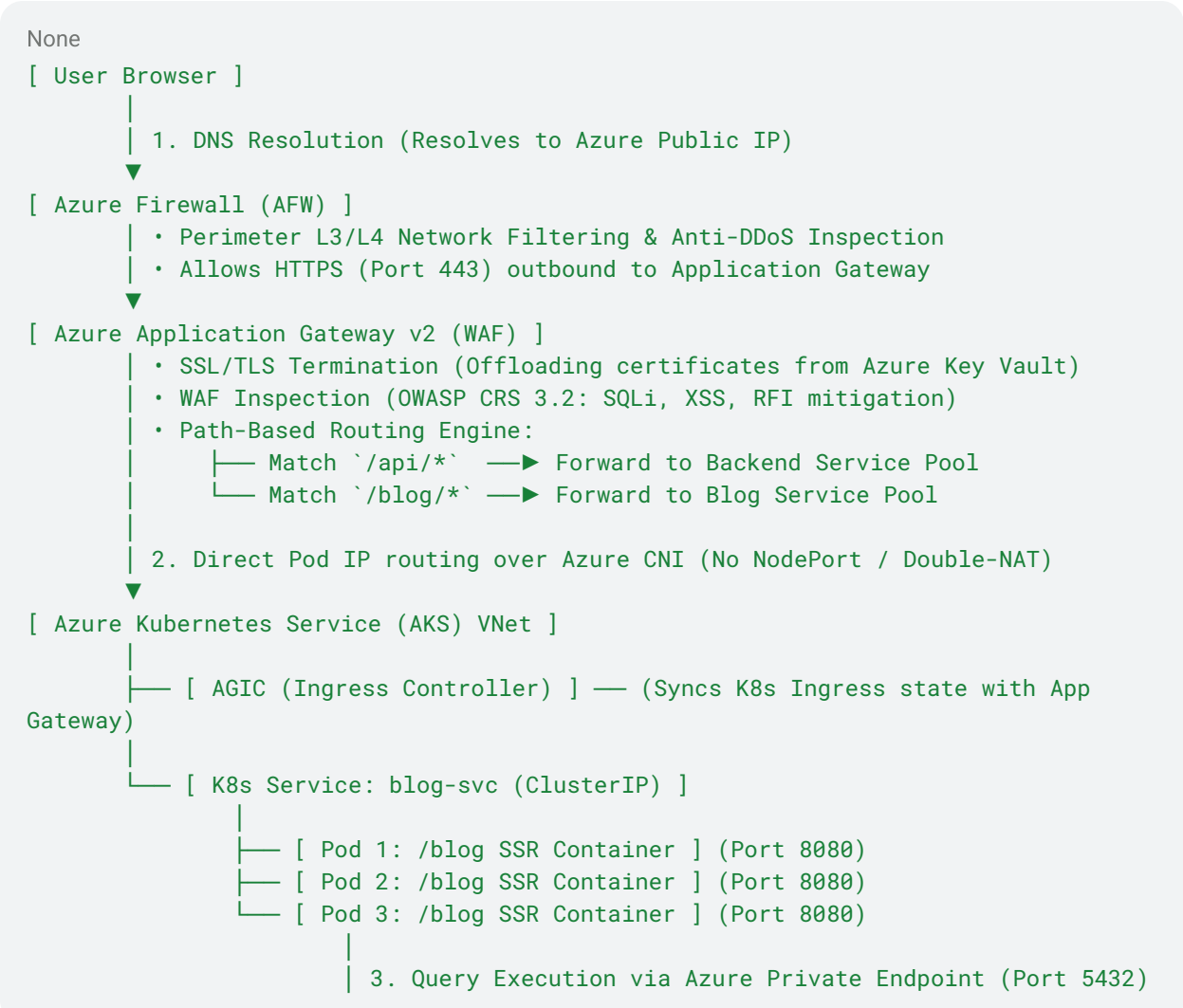
None





1.2 The Request Journey of `google.com/blog`

When a client enters `https://google.com/blog`, the request traverses perimeter security, layer-7 load balancing, cluster ingress, and internal networking before reaching the database:



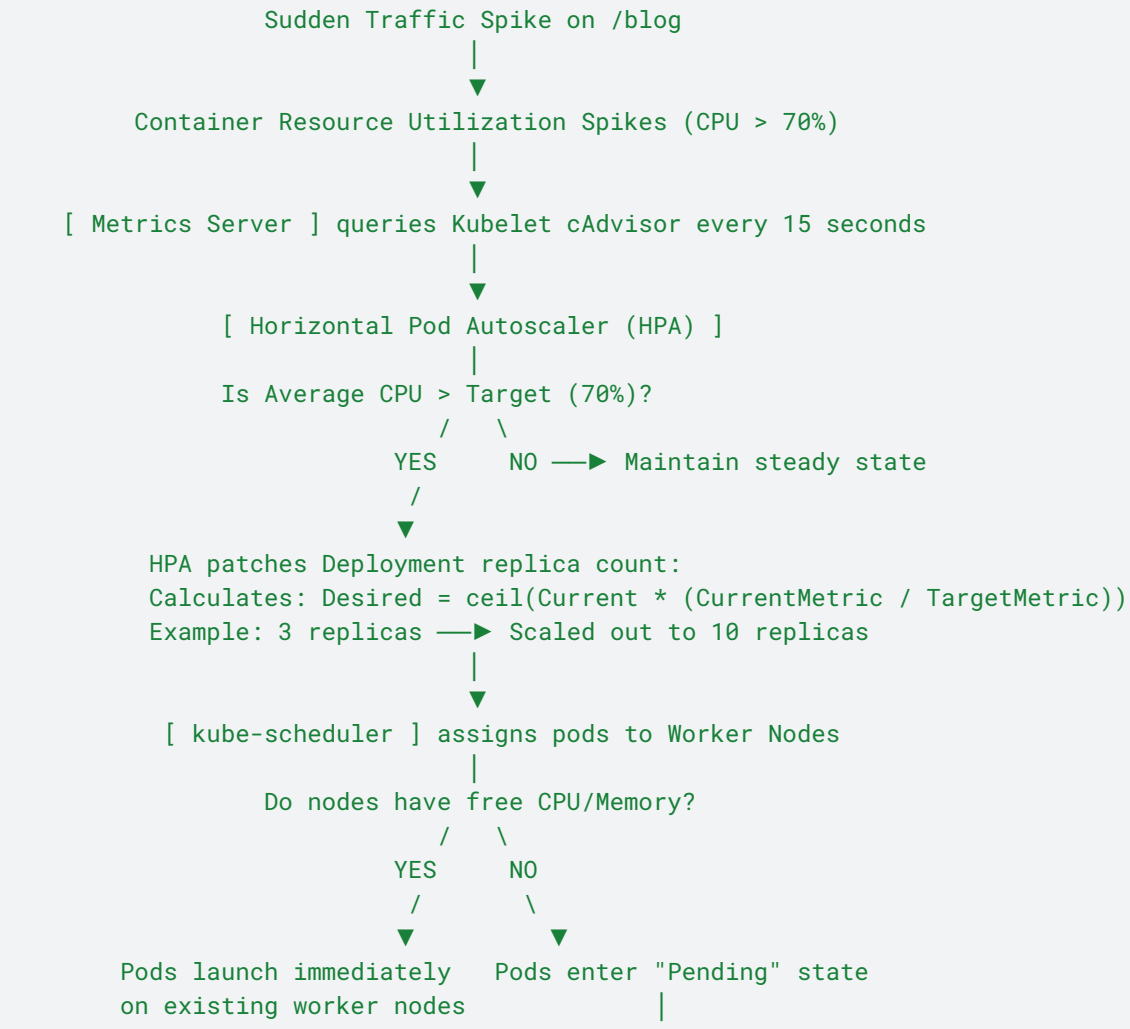
```
                ▼
[ Azure Database for PostgreSQL Flexible Server ]
(Private IP: 10.240.2.X - Isolated from the Public Internet)
```

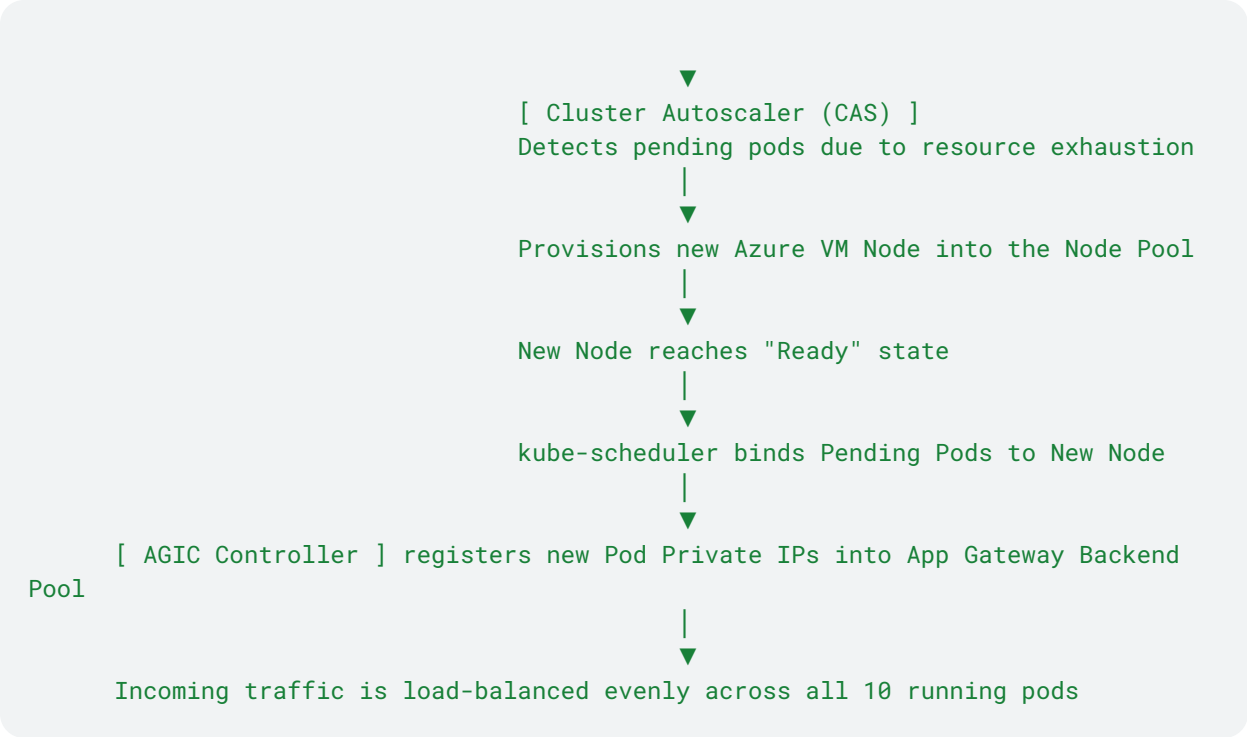
Module 2: Kubernetes Traffic Ingress & Pod Autoscaling Mechanics

2.1 Dynamic Autoscaling Workflow Under High Traffic

When incoming requests to `/blog` increase significantly, the Horizontal Pod Autoscaler (HPA) and the Cluster Autoscaler (CAS) scale both workloads and compute nodes.

None





2.2 Horizontal Pod Autoscaler Manifest (`hpa.yaml`)

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: blog-workload-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: blog-workload-deployment
  minReplicas: 3
  maxReplicas: 15
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
```

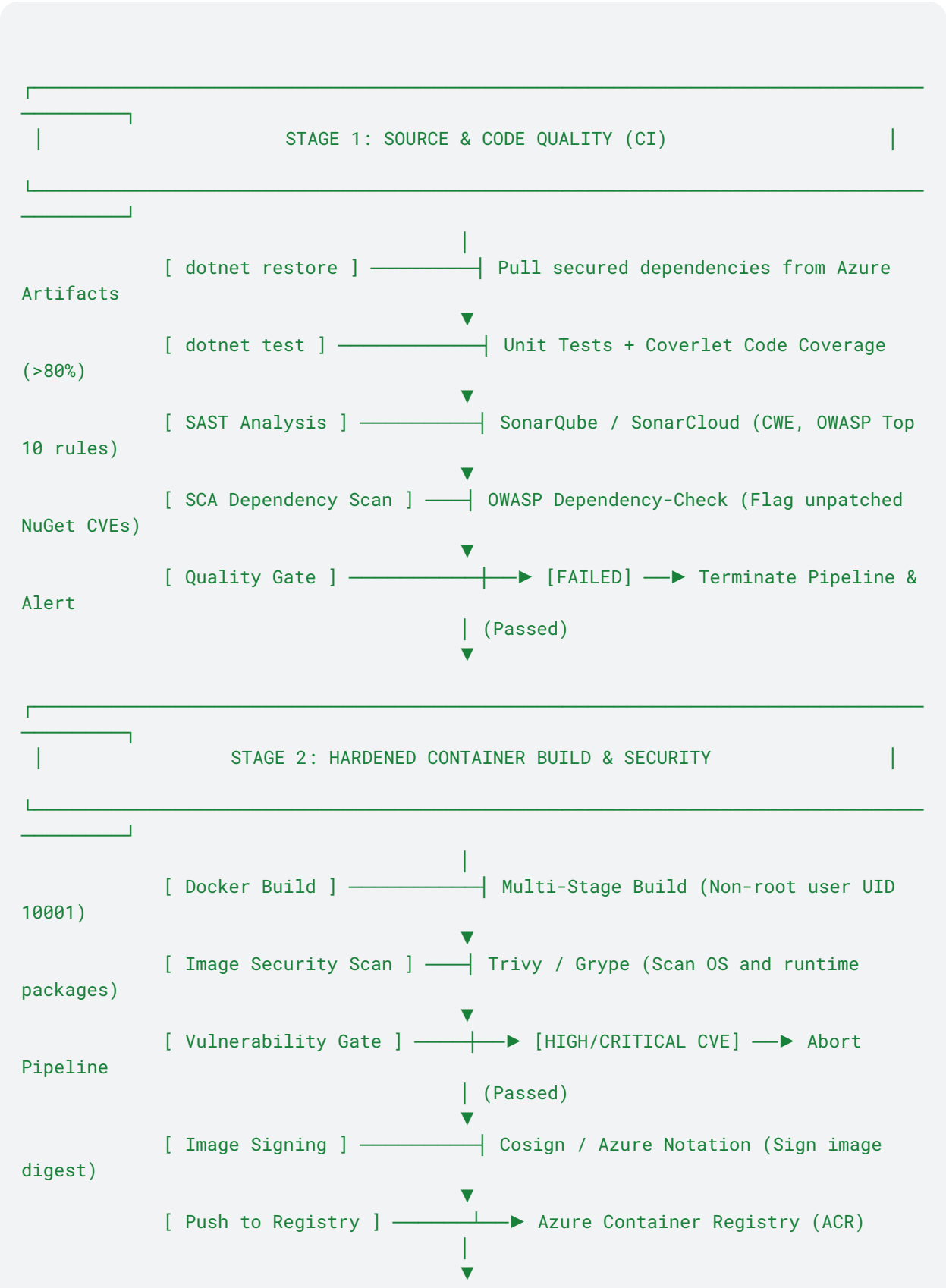
```
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300 # 5-minute cooldown to prevent
flapping
      policies:
      - type: Percent
        value: 50
        periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 0 # Immediate scale up on traffic
burst
      policies:
      - type: Percent
        value: 100
        periodSeconds: 15
```

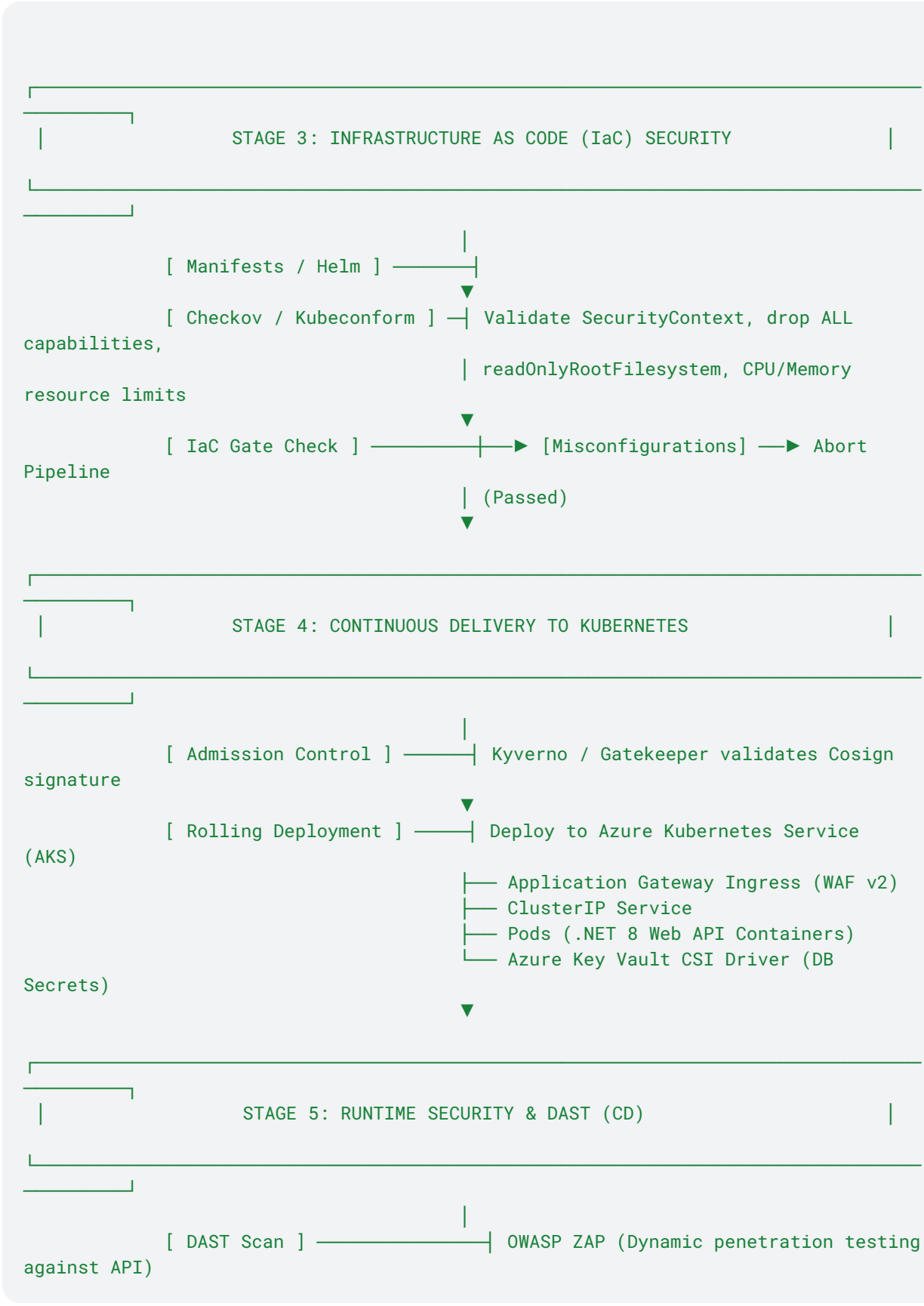
Module 3: End-to-End DevSecOps Delivery Pipeline for .NET Applications

3.1 .NET DevSecOps Architecture Flowchart

None

```
┌──────────────────────────────────────────────────────────────────────────────────────────┐
 │                    DEVELOPER WORKSTATION (Shift-Left)                    │
└──────────────────────────────────────────────────────────────────────────────────────────┘
                                          │
            [ IDE / Pre-commit ] ─────────┤ 1. Gitleaks / Talisman (Prevent
credential leaks)
                                          │ 2. Roslyn Analyzers & SonarLint (Local
static check)
                                          ▼
```


STAGE 1: SOURCE & CODE QUALITY (CI)
[dotnet restore] Pull secured dependencies from Azure Artifacts
[dotnet test] Unit Tests + Coverlet Code Coverage (>80%)
[SAST Analysis] SonarQube / SonarCloud (CWE, OWASP Top 10 rules)
[SCA Dependency Scan] OWASP Dependency-Check (Flag unpatched NuGet CVEs)
[Quality Gate]
[FAILED] Terminate Pipeline & Alert
(Passed)
STAGE 2: HARDENED CONTAINER BUILD & SECURITY
[Docker Build] Multi-Stage Build (Non-root user UID 10001)
[Image Security Scan] Trivy / Grype (Scan OS and runtime packages)
[Vulnerability Gate]
[HIGH/CRITICAL CVE] Abort Pipeline
(Passed)
[Image Signing] Cosign / Azure Notation (Sign image digest)
[Push to Registry]
Azure Container Registry (ACR)


STAGE 3: INFRASTRUCTURE AS CODE (IaC) SECURITY
[Manifests / Helm]
[Checkov / Kubeconform] Validate SecurityContext, drop ALL capabilities, readOnlyRootFilesystem, CPU/Memory resource limits
[IaC Gate Check]
[Misconfigurations]
Abort Pipeline
(Passed)
STAGE 4: CONTINUOUS DELIVERY TO KUBERNETES
[Admission Control] Kyverno / Gatekeeper validates Cosign signature
[Rolling Deployment] Deploy to Azure Kubernetes Service (AKS)
Application Gateway Ingress (WAF v2)
ClusterIP Service
Pods (.NET 8 Web API Containers)
Azure Key Vault CSI Driver (DB Secrets)
STAGE 5: RUNTIME SECURITY & DAST (CD)
[DAST Scan] OWASP ZAP (Dynamic penetration testing against API)

```
                                        ▼
            [ Runtime Threat Ops ] ─────┤ Falco / Microsoft Defender for
Containers
                                        | (Monitors syscalls, unexpected shell
executions)
```

3.2 Hardened .NET Multi-Stage Dockerfile

```dockerfile
# Stage 1: Build & Restore
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

COPY ["DemoApp.csproj", "./"]
RUN dotnet restore "DemoApp.csproj"

COPY . .
RUN dotnet build "DemoApp.csproj" -c Release -o /app/build

# Stage 2: Publish compiled DLLs
FROM build AS publish
RUN dotnet publish "DemoApp.csproj" -c Release -o /app/publish
/p:UseAppHost=false

# Stage 3: Minimal, Hardened Runtime Container
FROM mcr.microsoft.com/dotnet/aspnet:8.0-alpine AS final
WORKDIR /app

# Security: Create non-root system user and group
RUN addgroup -S appgroup && adduser -S appuser -G appgroup -u 10001
USER 10001:10001

COPY --from=publish --chown=10001:10001 /app/publish .

ENV ASPNETCORE_HTTP_PORTS=8080
EXPOSE 8080

ENTRYPOINT ["dotnet", "DemoApp.dll"]
```

3.3 Azure DevOps Pipeline for .NET (`azure-pipelines.yml`)

```yaml
trigger:
  branches:
    include:
      - main

variables:
  imageName: 'dotnet-secure-api'
  acrRegistry: 'myenterpriseacr.azurecr.io'
  tag: '$(Build.BuildId)'

stages:
- stage: CodeQualityAndSecurity
  displayName: 'Code Quality, SAST & SCA'
  jobs:
  - job: Analyze
    pool:
      vmImage: 'ubuntu-latest'
    steps:
    - task: UseDotNet@2
      inputs:
        packageType: 'sdk'
        version: '8.x'

    - task: SonarQubePrepare@5
      inputs:
        SonarQube: 'SonarQube-Connection'
        scannerMode: 'MSBuild'
        projectKey: 'dotnet-secure-api'
        projectName: 'dotnet-secure-api'

    - script: dotnet restore
      displayName: 'Restore Packages'

    - script: dotnet build --configuration Release --no-restore
      displayName: 'Build Solution'

    - script: dotnet test --configuration Release --no-build
--collect:"XPlat Code Coverage"
      displayName: 'Execute Tests'
```

```
    - task: SonarQubeAnalyze@5
    - task: SonarQubePublish@5
      inputs:
        pollingTimeoutSec: '300'

    - task: dependency-check-build-task@6
      displayName: 'OWASP Dependency-Check (SCA)'
      inputs:
        projectName: 'DemoApp'
        scanPath: '$(Build.SourcesDirectory)'
        format: 'HTML,JSON'
        failOnCVSS: '7'

- stage: ContainerSecurity
  displayName: 'Docker Build & Vulnerability Scan'
  dependsOn: CodeQualityAndSecurity
  jobs:
  - job: BuildAndScan
    pool:
      vmImage: 'ubuntu-latest'
    steps:
    - task: Docker@2
      displayName: 'Build Image'
      inputs:
        command: build
        repository: $(imageName)
        dockerfile: '**/Dockerfile'
        tags: $(tag)

    - script: |
        curl -sfL
https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/inst
all.sh | sh -s -- -b /usr/local/bin
        trivy image --exit-code 1 --severity CRITICAL,HIGH
$(imageName):$(tag)
      displayName: 'Trivy Scan: Abort on High/Critical CVEs'

    - task: Docker@2
      displayName: 'Push to Azure Container Registry'
      inputs:
        command: push
        containerRegistry: 'AzureACR-Connection'
        repository: $(imageName)
```

```
        tags: $(tag)

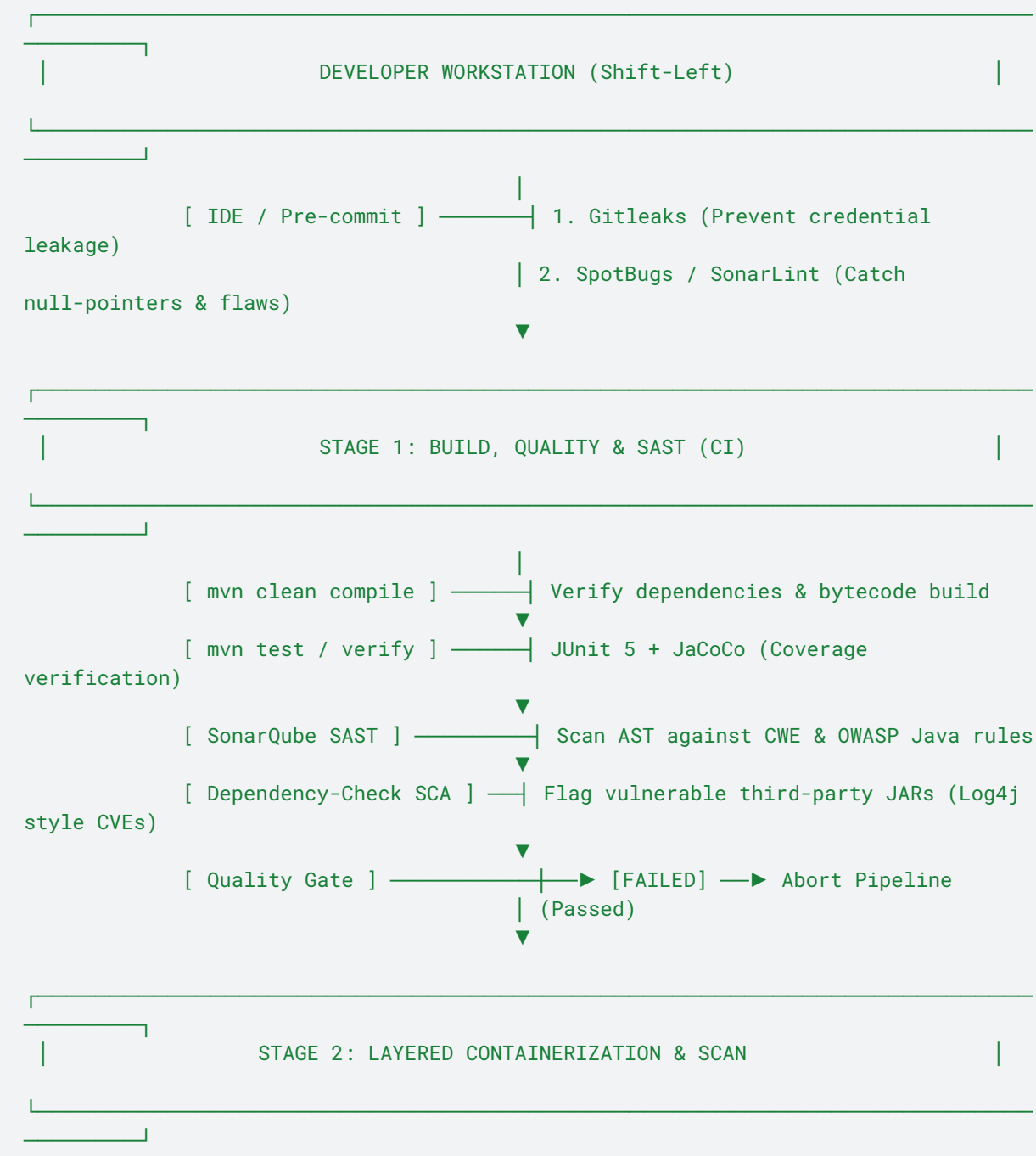
- stage: DeployToAKS
  displayName: 'Deploy to Kubernetes (Production)'
  dependsOn: ContainerSecurity
  jobs:
  - deployment: Deploy
    environment: 'production'
    pool:
      vmImage: 'ubuntu-latest'
    strategy:
      runOnce:
        deploy:
          steps:
          - script: |
              curl -s
https://raw.githubusercontent.com/yannh/kubeconform/master/scripts/ins
tall.sh | sh
              ./bin/kubeconform -strict manifests/
            displayName: 'Validate Manifests (Kubeconform)'

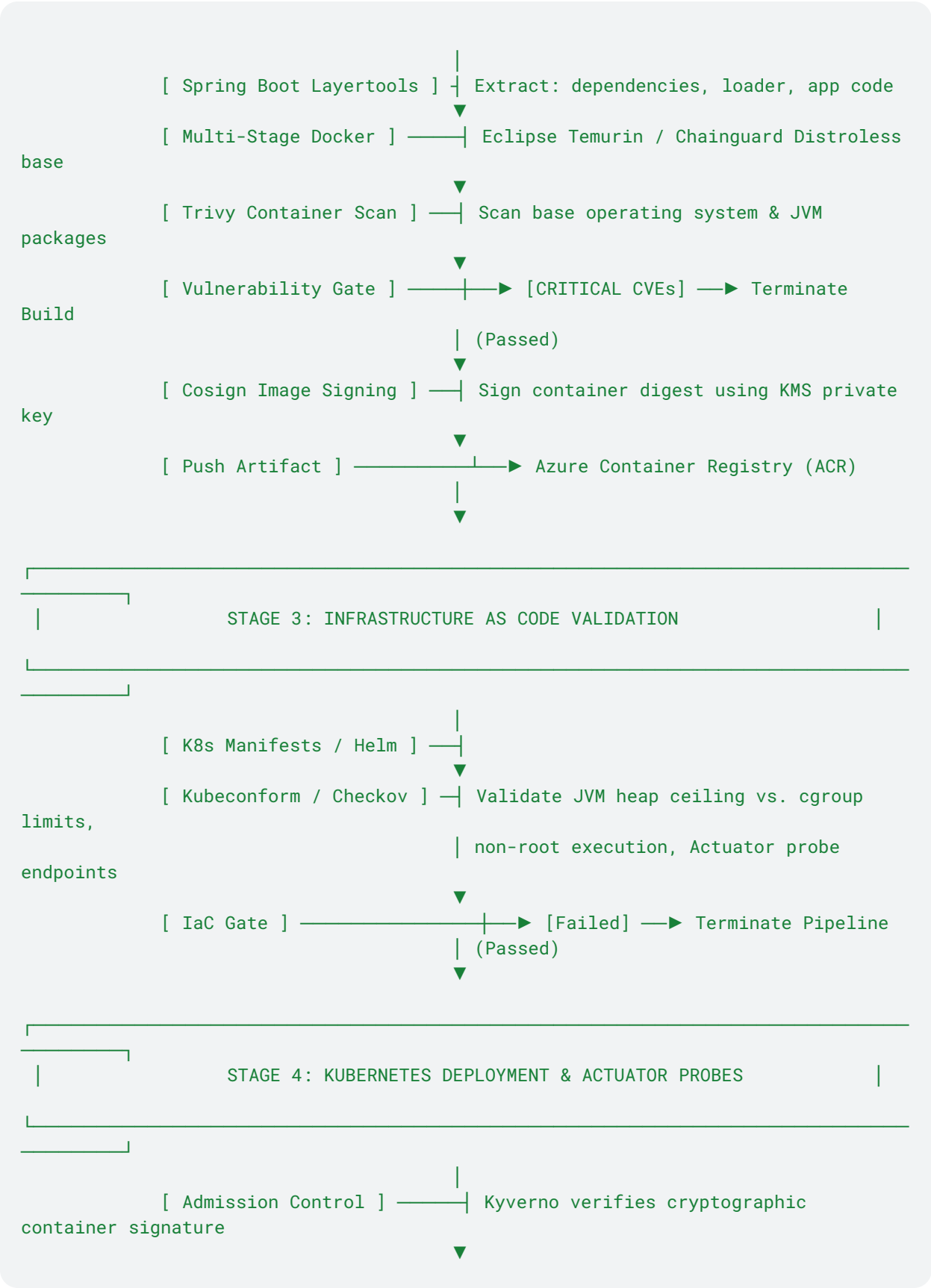
          - task: KubernetesManifest@1
            displayName: 'Deploy to AKS'
            inputs:
              action: 'deploy'
              connectionType: 'kubernetesServiceConnection'
              kubernetesServiceConnection: 'AKS-Production-Cluster'
              manifests: |
                manifests/deployment.yaml
                manifests/service.yaml
                manifests/ingress.yaml
              containers: |
                $(acrRegistry)/$(imageName):$(tag)
```

Module 4: End-to-End DevSecOps Delivery Pipeline for Java (Spring Boot) Applications

4.1 Java DevSecOps Architecture Flowchart

None




[Spring Boot Layertools] ┤ Extract: dependencies, loader, app code
[Multi-Stage Docker] ┤ Eclipse Temurin / Chainguard Distroless base
[Trivy Container Scan] ┤ Scan base operating system & JVM packages
[Vulnerability Gate] ┤ [CRITICAL CVEs] ▶ Terminate Build
(Passed)
[Cosign Image Signing] ┤ Sign container digest using KMS private key
[Push Artifact] ▶ Azure Container Registry (ACR)
STAGE 3: INFRASTRUCTURE AS CODE VALIDATION
[K8s Manifests / Helm]
[Kubeconform / Checkov] ┤ Validate JVM heap ceiling vs. cgroup limits,
non-root execution, Actuator probe endpoints
[IaC Gate] ┤ [Failed] ▶ Terminate Pipeline
(Passed)
STAGE 4: KUBERNETES DEPLOYMENT & ACTUATOR PROBES
[Admission Control] ┤ Kyverno verifies cryptographic container signature

```
            [ Deploy Workload to AKS ] ┤ Apply Kubernetes Resources
                                       ├── Application Gateway Ingress (WAF v2)
                                       ├── Spring Boot Pods (with Actuator
Probes)
                                       └── Workload Identity connection to Azure
PostgreSQL
                                       ▼

┌──────────────────────────────────────────────────────────────────────────────
──────────┐
 |                STAGE 5: RUNTIME PROTECTION & DAST                          |
└──────────────────────────────────────────────────────────────────────────────
──────────┘
                                       |
            [ DAST Scan ] ─────────────┤ OWASP ZAP validates runtime API
endpoints
                                       ▼
            [ Runtime Threat Monitor ] ┤ Falco detects remote code execution /
shell injections
```

4.2 Optimized Layered Spring Boot Dockerfile

```dockerfile
# Stage 1: Extract layers using Spring Boot layertools
FROM eclipse-temurin:21-jre-alpine AS builder
WORKDIR /application
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} application.jar
RUN java -Djarmode=layertools -jar application.jar extract

# Stage 2: Hardened Runtime Container
FROM eclipse-temurin:21-jre-alpine AS final
WORKDIR /application

# Run as non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup -u 10001
USER 10001:10001

# Copy layers separately to maximize Docker cache utilization
COPY --from=builder --chown=10001:10001 /application/dependencies/ ./
COPY --from=builder --chown=10001:10001
/application/spring-boot-loader/ ./
```

```
COPY --from=builder --chown=10001:10001
/application/snapshot-dependencies/ ./
COPY --from=builder --chown=10001:10001 /application/application/ ./

# Configure JVM for Kubernetes container awareness (prevents OOM
kills)
ENV JAVA_TOOL_OPTIONS="-XX:+UseContainerSupport
-XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=50.0
-Djava.security.egd=file:/dev/./urandom"

EXPOSE 8080

ENTRYPOINT ["java",
"org.springframework.boot.loader.launch.JarLauncher"]
```

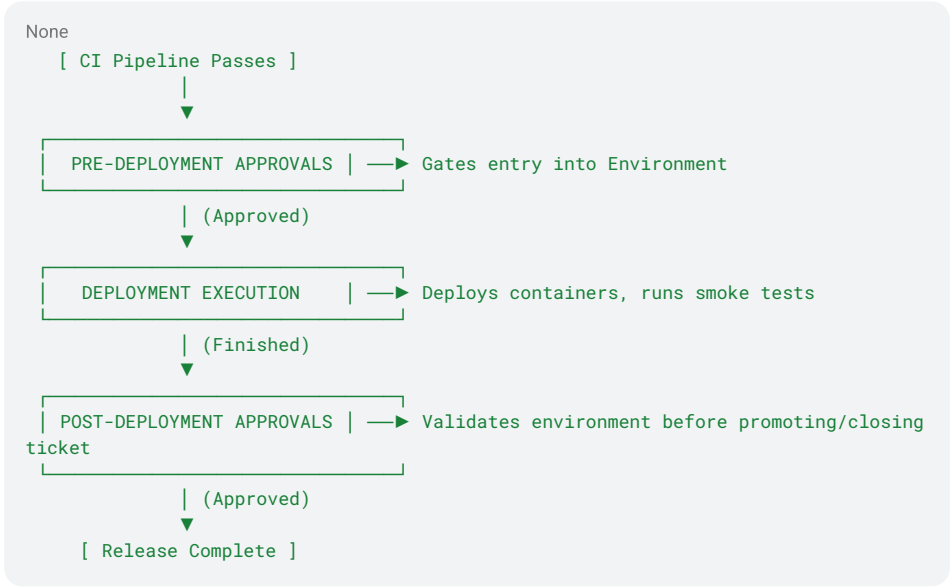
4.3 Hardened Kubernetes Deployment for Java (`deployment.yaml`)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: spring-boot-api
  namespace: production
  labels:
    app: spring-boot-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: spring-boot-api
  template:
    metadata:
      labels:
        app: spring-boot-api
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        runAsGroup: 10001
        fsGroup: 10001
        seccompProfile:
```

```
          type: RuntimeDefault
      containers:
      - name: spring-app
        image: myenterpriseacr.azurecr.io/spring-boot-api:latest
        imagePullPolicy: IfNotPresent
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities:
            drop:
            - ALL
        ports:
        - containerPort: 8080
        resources:
          requests:
            cpu: "250m"
            memory: "512Mi"
          limits:
            cpu: "1000m"
            memory: "1Gi" # JVM MaxRAMPercentage=75% leaves 256MiB for
Metaspace/Native overhead
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8080
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8080
          initialDelaySeconds: 20
          periodSeconds: 5
        volumeMounts:
        - name: tmp-volume
          mountPath: /tmp
      volumes:
      - name: tmp-volume
        emptyDir: {}
```

Module 5: Pipeline Release Governance: Approvals Framework

5.1 Deployment Approval Stages



5.2 Approver Configuration Matrix

Approval Gate	Recommended Count	Typical Approvers	Primary Purpose
Dev / QA (Pre)	**0** (Automated)	None	Fully automated pipeline for rapid feedback.
Staging (Pre)	**1 Approver**	QA Lead / Dev Lead	Verifies functional testing and sign-off from lower environments.
Production (Pre)	**1 to 2 Approvers**	1. Tech Lead / Release Manager 2. Change Advisory Board (CAB)	Verifies production change windows, maintenance notices, and rollback readiness.

Approval Gate	Recommended Count	Typical Approvers	Primary Purpose
Production (Post)	**0 to 1 Approver**	Product Owner / QA Automation Lead	Verifies live sanity checks and business workflows before closing the release ticket.

5.3 Approval Evaluation Logic Options

1. **Any One User ("OR" Logic):** A pool of 3 to 5 senior engineers is designated; as soon as **any single individual** approves, the deployment proceeds. This avoids deployment bottlenecks caused by absence.
2. **All Users in Any Order ("AND" Logic):** Requires signatures from multiple distinct teams (e.g., both the **Security Lead** AND the **QA Manager**) before the deployment unlocks.
3. **Automated Business Hour Gates:** Enforces execution only within approved maintenance windows (e.g., Tuesday at 02:00 UTC).